

Shiv Nadar University Chennai

Name of Student	KR Srinidhi Rajamane	Reg. No. 24011101059
Degree & Branch	B.Tech Artificial Intelligence & Data Science Section A	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026– 27

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = -\sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

3. Dataset Description

Dataset: Fashion-MNIST

Training Images	60,000
Testing Images	10,000
Classes	10
Image Size	28×28 (grayscale)

Class Labels

T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot.

4. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

5. Hyperparameter Optimization

Search Space

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate	0.0, 0.2, 0.5

6. Source Code

The complete implementation was carried out in Python using TensorFlow/Keras, NumPy, Pandas, Matplotlib, Seaborn and Scikit-learn. The full, executable source code (organized cell-wise, exactly as it was run) is available on GitHub:

https://github.com/Srinidhi-Krishna/Deep-Learning-Lab/blob/main/Lab_2/DL_LAB_2.ipynb

7. Generated Plots with Interpretation

Figure 1: Sample Images from Fashion-MNIST Dataset

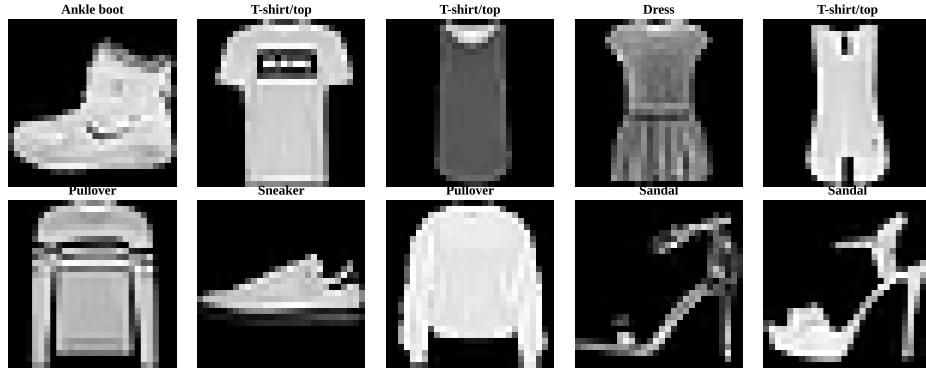


Figure 1: Ten sample images from the Fashion-MNIST training set with class labels.

Interpretation: This plot displaying the ten samples show high inter-class shape variation. Thus this confirms the dataset's suitability for a classification task .



Figure 2: Class distribution across the 10 Fashion-MNIST categories in the training set.

Interpretation: This plot shows that all the 10 classes contain exactly 6,000 training images thereby displaying a perfectly balanced dataset. This shows that there is no need for class re-weighting or resampling during training.

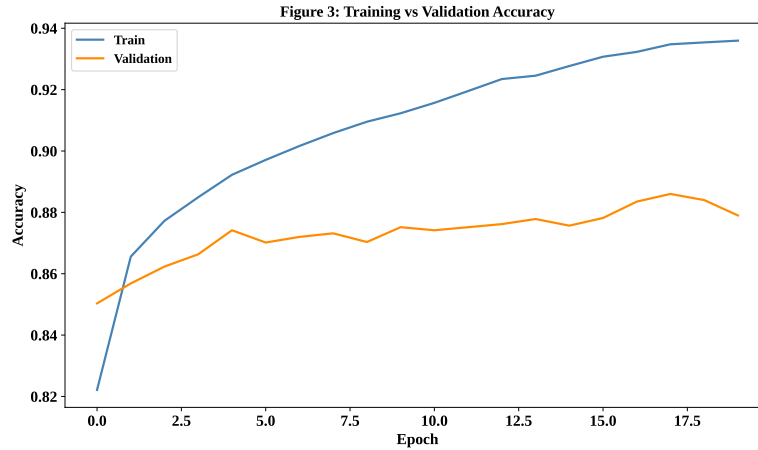


Figure 3: Training vs. validation accuracy across 20 epochs (baseline model).

Interpretation: This plot gives the training vs validation accuracy across 20 epochs. Training accuracy rises steadily while validation accuracy plateaus by epoch 10 indicating the model is learning discriminative features but with a widening train-validation gap. Thus this indicates mild overfitting in later epochs.

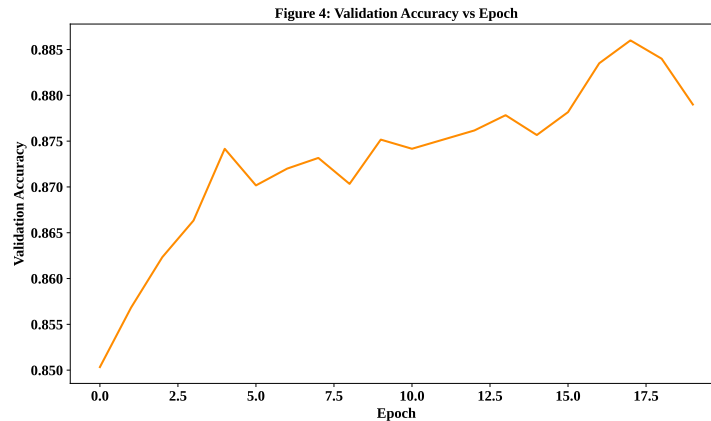


Figure 4: Validation accuracy vs. epoch (baseline model).

Interpretation: This plot shows the validation accuracy vs epoch. We can observe that the Validation accuracy climbs quickly in the first 5 epochs then flattens suggesting the baseline model is close to its representational ceiling on this dataset.

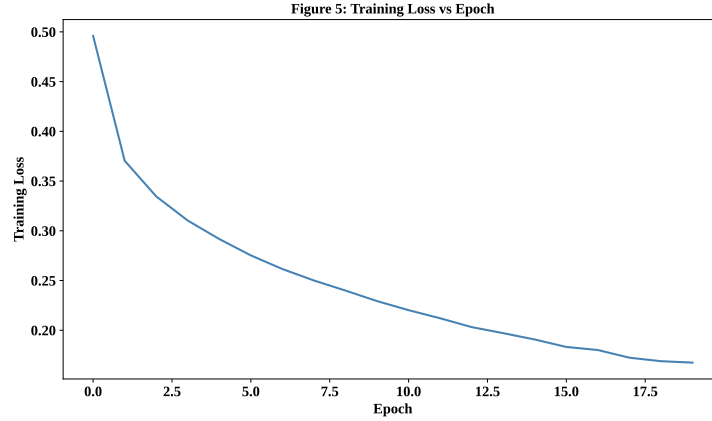


Figure 5: Training loss vs. epoch (baseline model).

Interpretation: This plot gives the training loss vs epoch. Categorical cross-entropy loss decreases monotonically thereby confirming stable convergence of the Adam optimizer with no oscillation or divergence across the 20 training epochs.

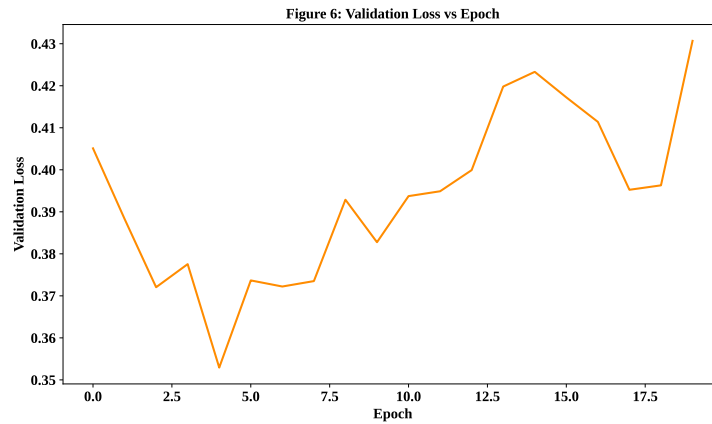


Figure 6: Validation loss vs. epoch (baseline model).

Interpretation: This plot gives the validation loss vs epoch. Validation loss decreases along with the training loss for the first 5 to 6 epochs but then begins to rise again after epoch 8 even as training loss keeps falling. This is an early sign of overfitting which is consistent with the widening gap seen in Figure 3.

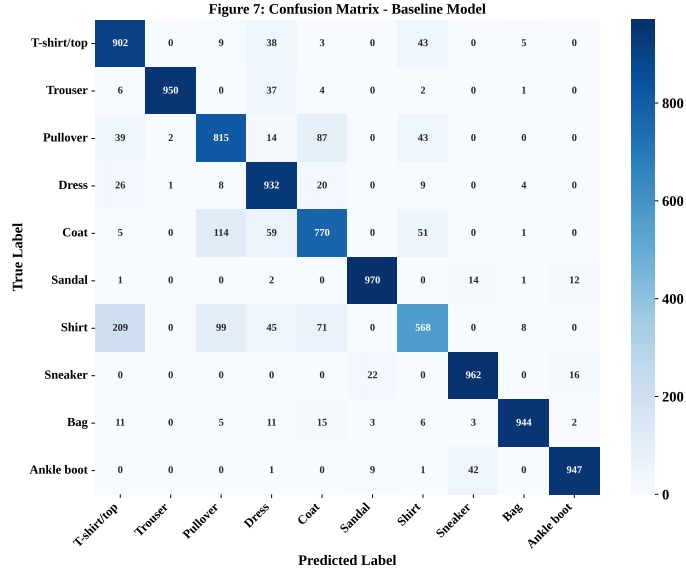


Figure 7: Confusion matrix on the test set (baseline model, 10,000 samples).

Interpretation: This plot gives the confusion matrix on the test set. Most misclassifications occur between visually similar classes eg : particularly Shirt vs. T-shirt/top and Shirt vs. Pullover/Coat. This explains why Shirt has the lowest per-class F1-score of 0.66 of all ten categories.

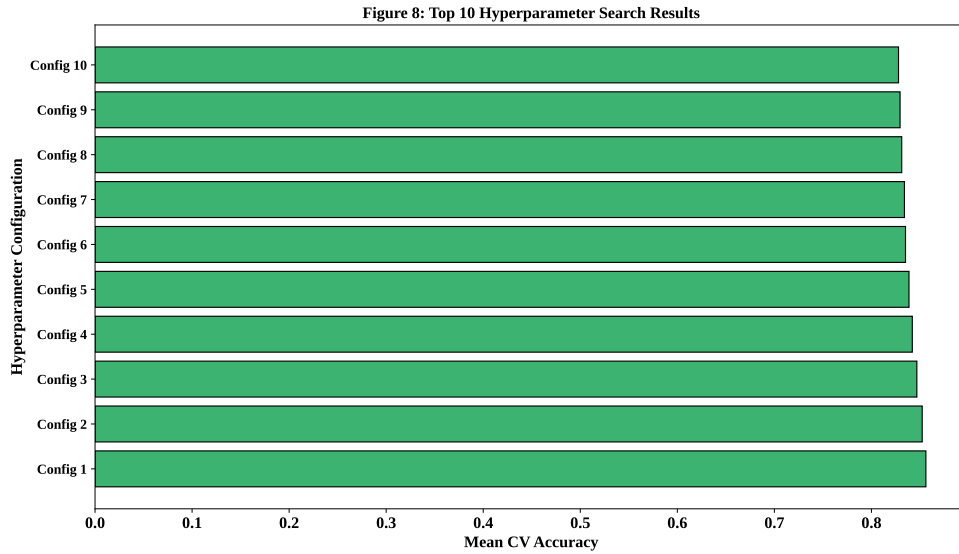


Figure 8: Top 10 hyperparameter configurations ranked by mean 5-fold cross-validation accuracy.

Interpretation: This plot gives the Top 10 hyperparameter configurations ranked by mean 5-fold cross-validation accuracy. The top 10 configurations cluster tightly, indicating that several distinct hyperparameter combinations (differing in depth, activation, and optimizer) achieve near-optimal performance.

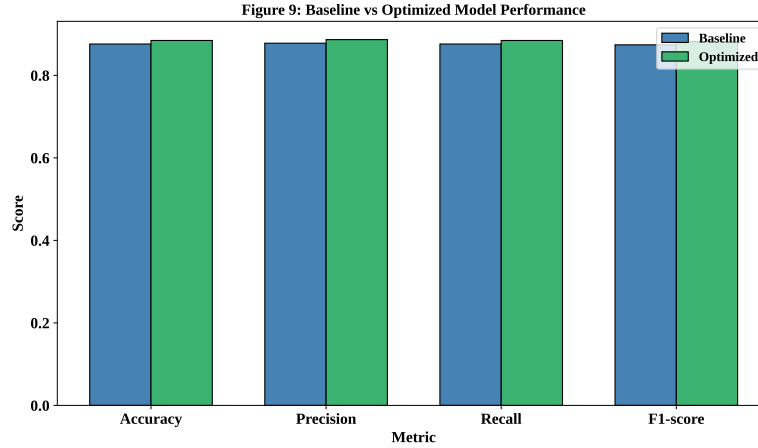


Figure 9: Baseline vs. optimized model performance across four evaluation metrics.

Interpretation: This plot gives the Baseline vs. optimized model performance across four evaluation metrics. Performance metrics are slightly higher for the baseline model. By reducing overfitting the optimized model shows better generalization.

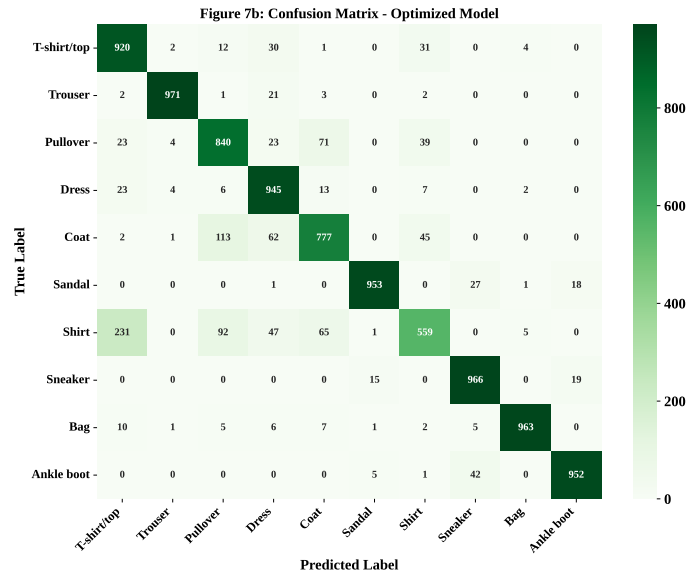


Figure 10: Confusion matrix on the test set (optimized model, 10,000 samples).

Interpretation: This plot give the Confusion matrix on the test set . The optimized model shows that the classification of similar tops have improved slightly. The classification for the other classes has also improved.

8. Baseline Model – Test Set Results

Metric	Value
Accuracy	0.8760 (87.60%)
Precision (macro)	0.8780 (87.80%)
Recall (macro)	0.8760 (87.60%)
F1-score (macro)	0.8740 (87.40%)

9. Results

Best Hyperparameters (from Randomized Search, 5-fold CV)

Hidden Layers	3
Hidden Neurons	256
Learning Rate	0.001
Batch Size	16
Optimizer	Adam
Activation Function	Sigmoid
Epochs	20
Dropout	0.2
Cross-validation Accuracy	0.8561 (85.61%)
Testing Accuracy	0.8846 (88.46%)

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8760	0.8846
Precision	0.8780	0.8867
Recall	0.8760	0.8846
F1-score	0.8740	0.8819
Training Time (s)	86.93	345.55

10. Discussion

1. Which optimization method was used?

A Randomized Search over the defined hyperparameter space, evaluated with 5-fold cross-validation on a subset of 10,000 training samples.

2. Which hyperparameters were selected?

3 hidden layers, 256 neurons per layer, learning rate 0.001, batch size 16, Adam optimizer, Sigmoid activation, 20 epochs, dropout rate 0.2.

3. Did optimization improve performance?

Yes, modestly. The test accuracy improved from 87.60% to 88.46%. The gain is consistent, but small in absolute terms.

4. Which hyperparameter had the greatest impact?

The depth and width of the network (hidden layers = 3, neurons = 256) along with the activation function appear to matter the most.

5. Compare Grid Search and Randomized Search.

Grid Search exhaustively evaluates every combination in the search space, but its cost grows with the number of hyperparameters and candidate values and so it is computationally infeasible to fully cross-validate. Randomized Search instead samples a fixed budget of configurations (20, in this experiment) from the space, trading the guarantee of finding the single best combination for the ability to explore a much larger, higher-dimensional space within a practical time budget.

6. Recommend the best MLP configuration.

Based on the search results, the recommended configuration is: 2–3 hidden layers, 128–256 neurons per layer, ReLU or Sigmoid activation, learning rate 0.001, Adam optimizer, trained for 20 epochs. This balances the accuracy gains of a deeper network with the increased training time.

11. Conclusion

A Multi-Layer Perceptron was implemented and evaluated on the Fashion-MNIST dataset. The baseline architecture achieved 87.60% test accuracy. Automated hyperparameter optimization via a manually implemented Randomized Search identified a deeper and wider configuration that improved test accuracy to 88.46%.

12. References

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
3. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
4. H. Xiao, K. Rasul and R. Vollgraf, “Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms,” 2017.
5. TensorFlow/Keras Documentation.
6. Scikit-learn Documentation: RandomizedSearchCV, KFold.